



Instituto Infnet

Quando não existe API

Projeto de Bloco: Inteligência Artificial Aplicada

Aula 7 · Etapa 4 · Extração de conteúdo web

Prof. Renan Santos



O pedido que não tem endpoint

A gestora abriu o painel interativo — e voltou com uma pergunta diferente.

- **"Quando eu apresento no conselho, me perguntam: e o que está acontecendo AGORA?"**
População de 2025 não responde a isso.
Ela quer o que a imprensa está publicando, ao lado dos indicadores.
- **Você faz o que aprendeu na Aula 2: procura a API**
E descobre o que quase toda coleta descobre: NÃO EXISTE.
- **O IBGE publica população em JSON. Notícia, ninguém publica**
Nenhum site vai montar um endpoint só porque seria conveniente para você.

► A pergunta de hoje: como transformar uma página feita para OLHOS em uma tabela?

Duas fontes, dois contratos

O que muda da Aula 2 para hoje — e a última linha é a perigosa.

Duas fontes, dois contratos muito diferentes

API · o que você já fez (Aula 2)

```
{ "localidade": {"id": "35"},  
  "serie": {"2025": "46081801"} }
```

Formato JSON — feito para PROGRAMAS

Estabilidade contrato versionado

Permissão implícita no endpoint público

Quando quebra erro claro: 404, 500

Página web · hoje (Aula 7)

```
<a href="/noticia/2026-09/...">  
  <h2>Depois da COP17...</h2></a>
```

Formato HTML — feito para OLHOS

Estabilidade muda quando o dono quiser

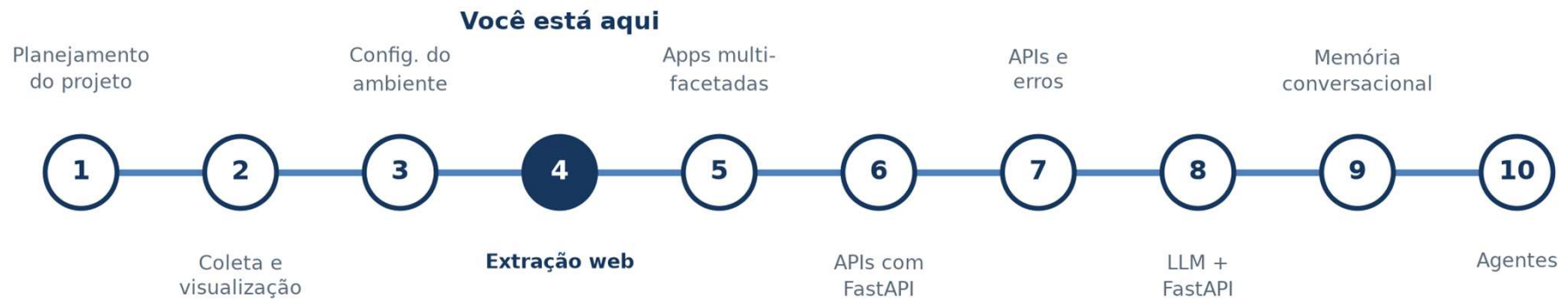
Permissão robots.txt + licença

Quando quebra **SILÊNCIO: lista vazia**

A linha perigosa é a última: a API avisa quando quebra; a página, não.

Onde estamos no Projeto de Bloco

Etapa 4, primeira metade: subcompetência 2.4. E o TP2 FECHA nesta etapa.



Objetivos da aula

Cinco capacidades — e a primeira não é sobre programar.

- **Verificar se você PODE raspar**
robots.txt, Crawl-delay e licença — antes do código.
- **Baixar com requests**
Ler o status HTTP e gravar o snapshot antes de processar.
- **Extrair com BeautifulSoup**
find · find_all · select · get_text.
- **Limpar e gravar**
CSV/TXT em data/, com guarda contra resultado vazio.
- **Justificar a arquitetura**
Por que a coleta roda FORA do app.

No TP2 — a etapa que FECHA

O enunciado é literal. Leia com a turma.

- **"Utilize a ferramenta BeautifulSoup para extrair conteúdo de páginas web."**
É a subcompetência 2.4 — o que fazemos hoje.
- **"Execute esses códigos SEPARADAMENTE e armazene em CSV e/ou TXT em data/."**
O enunciado já manda separar coleta de uso. Vamos ver por quê — e a razão é melhor do que 'porque o enunciado mandou'.
- **"Posteriormente, utilize esses dados para alimentar a interface."**
Isso é a Aula 8: upload, cache e estado. Aí o TP2 fecha.

► **Etapa 3 abriu a Competência 2. A Etapa 4 fecha — e o TP2 vai junto.**

1 · A primeira pergunta não é técnica

Três perguntas antes da primeira linha de código

Nenhuma delas se responde no editor.

- **Posso baixar? → `site.com/robots.txt`**

Pedido explícito do dono do site, num formato legível por máquina.

"Seguir as diretivas do robots.txt é DEVER ÉTICO de todo desenvolvedor" [Chapagain, p100]

- **De quanto em quanto tempo? → Crawl-delay**

O ritmo que ele aceita. Vai DECIDIR a arquitetura do seu programa.

- **Posso publicar o que baixei? → a LICENÇA do conteúdo**

Pergunta diferente, e independente: direito autoral não está no robots.txt.

Dá para ter permissão de baixar e não ter permissão de republicar.

► robots.txt responde 'posso BAIXAR?'. A licença responde 'posso PUBLICAR?'.

A biblioteca padrão já sabe ler robots.txt

passo01_permissao.py

```
from urllib.robotparser import RobotFileParser # vem COM o Python

regras = RobotFileParser()
regras.parse(requests.get(f"{SITE}/robots.txt").text.splitlines())

regras.can_fetch(AGENTE, SITE + "/meio-ambiente") # True
regras.can_fetch(AGENTE, SITE + "/cron.php")      # False
regras.crawl_delay(AGENTE)                        # 10

# Zero dependência nova. A pergunta é anterior a qualquer ferramenta.
# É o mesmo caminho do livro-texto: [Chapagain, p102]
```



Os dois alvos que morreram aqui

História real deste material, de 02/09/2026. Nenhuma das duas mortes foi bug.

- **Alvo 1 · Agência IBGE Notícias → HTTP 403**

Mesma instituição da API que o painel já usa. Parecia perfeito.

Bloqueio de IP é a 2ª das três reações que o livro lista [Chapagain, p100].

- **Alvo 2 · ONU News, seção ODS → robots.txt: Disallow: */news/**

Tema perfeito, HTML limpo, HTTP 200. Ia funcionar.

O servidor teria entregue sem reclamar: robots.txt NÃO É CADEADO, é PEDIDO.

- **Alvo 3 · Agência Brasil (EBC) → 200, permitido, CC Atribuição 3.0**

Crawl-delay: 10 s. Reutilizável CITANDO a fonte — daí a coluna `fonte` no CSV.

► Três reações possíveis do site: CAPTCHA · bloqueio de IP · ação legal [Chapagain, p100].

Identifique-se de verdade

O User-Agent é a sua carteira de identidade na requisição.

- **AGENTE = "INFNET-PB-IA/1.0 (material didatico; contato: professor)"**

Quem é · para quê · como avisar se você estiver incomodando.

- **ANTI-PADRÃO: copiar a assinatura de um Chrome**

É o primeiro passo de quem decidiu ignorar o dono do site.

E é autodestrutivo: quando bloquearem, ninguém sabe a quem liberar.

- **Crawl-delay: 10 s muda o DESENHO do programa**

3 seções + 15 matérias = 18 requisições = 186 segundos, medidos.

Com esse número, raspar dentro de um app interativo é impossível.

► **Restrição externa não é obstáculo a contornar: é requisito a incorporar.**



2 · Do texto à estrutura

O caminho da coleta

Seis etapas — e a primeira não tem código nenhum.

O caminho da coleta — e ele não começa no código



↺ para cada link encontrado: repita as etapas 2-5 (crawling)

Pedir licença antes de pegar · baixar uma vez, processar muitas · limpar é metade do trabalho

Baixar uma vez — e LER o status

`passo02_baixar_e_guardar.py`

```
resposta = requests.get(URL, headers={"User-Agent": AGENTE}, timeout=30)

print(resposta.status_code)          # LEIA. Sempre.
resposta.raise_for_status()          # 403/404/500 param AQUI

resposta.encoding = resposta.apparent_encoding or "utf-8"
DESTINO.write_text(resposta.text, encoding="utf-8")  # o SNAPSHOT

# A partir daqui, o passo 3 lê ESTE arquivo.
# O servidor dos outros não é o seu disco.
```



O 403 que o BeautifulSoup engoliu sorrindo

O tropeço mais caro do build — meia hora perdida no seletor errado.

- **O script rodava sem erro e devolvia ZERO notícias**
Nenhuma exceção. Nenhum aviso. Lista vazia.
- **A causa: o 403 vem COM uma página de erro em HTML válido**
O BeautifulSoup processou aquilo alegremente, procurou os títulos e não achou.
- **A correção tem 24 caracteres: `resposta.raise_for_status()`**
Antes de parsear. Sempre.

► **Sem checar o status, você depura o seletor CERTO contra a página ERRADA.**

A sopa: quatro operações resolvem 90%

passo03_sopa.py

```
sopa = BeautifulSoup(html, "html.parser") # o parser vem com o Python
```

```
sopa.find("h1") # o PRIMEIRO que casar -> tag ou None
```

```
sopa.find_all("a", href=True) # TODOS -> lista
```

```
sopa.select("div.box-texto") # seletor CSS -> lista
```

```
tag.get_text(" ", strip=True) # só o texto -> str
```

```
# As duas regras que evitam metade dos bugs:
```

```
# find devolve None quando não acha [Chapagain, p135]
```

```
# find_all devolve LISTA VAZIA quando não casa [Chapagain, p137]
```

```
# .get_text() em None é AttributeError. PERGUNTE antes de usar.
```

Do link para o título — quem manda é o que você quer levar

passo03_sopa.py

```
for link in sopa.find_all("a", href=True):
    if "/noticia/" not in link["href"]:      # atributo = dicionário
        continue
    titulo = link.find(["h1", "h2", "h3"])    # o título está DENTRO do link
    if titulo is None:
        continue                            # link de imagem: ignore
    print(titulo.get_text(" ", strip=True), link["href"])

# 170 links na página -> 27 apontam para /noticia/ -> 5 manchetes distintas
# (a mesma matéria aparece 2x: destaque + lista)
```

Dois níveis — isto é web crawling

A listagem dá a manchete. A data exata e o texto estão dentro da matéria.

- **Nível 1 • página de seção**
5 manchetes + os links de cada uma.
- **Nível 2 • página da matéria**
h1 (título) · div.data (data) · div.conteudo-noticia (texto completo).
Seguir o link e repetir a extração: é isso que se chama crawling.
- **E exige pausa entre as requisições**
18 requisições × 10 s = 186 s. Medido, não estimado.

► O texto completo é a matéria-prima da nuvem de palavras que o TP2 pede (Aula 8).

Limpar é METADE do trabalho

passo04_limpar_e_gravar.py

```
# DOM imperfeito -> conteúdo incompleto -> limpeza ou rejeição [Chapagain, p129]
# o que sai da página:
#   'Rio terá dia de chuva\xa0e mar de ressaca'
#   'Publicado em   02/09/2026 - 07:02'
```

```
def limpar_texto(bruto):          # \xa0 = espaço que NÃO quebra linha
    return re.sub(r"\s+", " ", bruto.replace("\xa0", " ")).strip()
```

```
def limpar_data(bruto):          # ancorado no FORMATO, não na posição
    achado = re.search(r"(\d{2}/\d{2}/\d{4})\s*-\s*(\d{2}:\d{2})", bruto)
    return f"{achado.group(1)} {achado.group(2)}" if achado else ""
```

A guarda que salva a coleta anterior

Zero linhas é quase sempre seletor quebrado — não site vazio.

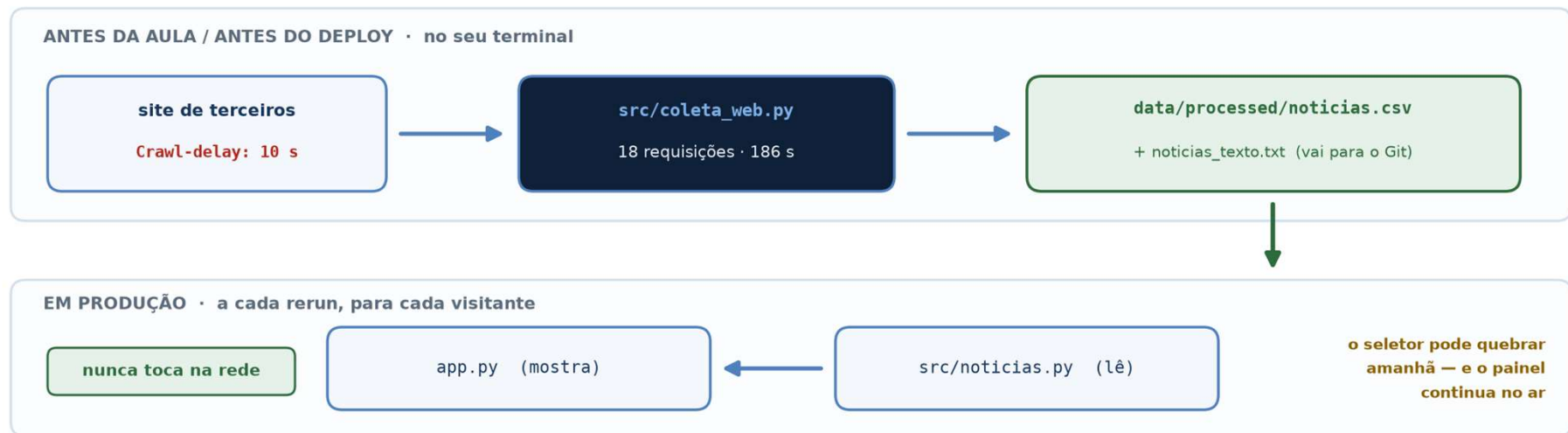
- **if not linhas: raise SystemExit('...NÃO vou gravar')**
find_all devolve "an empty list() object if the query doesn't match" [Chapagain, p137]:
seletor quebrado NÃO levanta erro. Sem a guarda, o CSV bom é sobrescrito.
- **newline="" no open() — sete caracteres**
Sem ele, no Windows, o csv escreve uma linha em branco entre cada registro.
- **O modo de falhar do scraping é o SILÊNCIO**
Quebra aparece na hora, com rastro. Engana você descobre tarde.

► Ler o status · guardar o snapshot · checar antes de gravar. Três hábitos simples.

A coleta é uma ETAPA, não um recurso do app

Por que `src/coleta_web.py` NÃO é importado pelo `app.py`.

A coleta é uma ETAPA, não um recurso do app



A fragilidade da raspagem não pode virar fragilidade do produto — e é o que o enunciado do TP2 manda fazer.

A razão maior — e ela não é o Crawl-delay

O Crawl-delay torna obrigatório. Isto torna certo.

- **O seletor VAI quebrar um dia**

A estrutura da página é escolha do dono, e ele não te avisa quando muda.

- **Se isso derrubar o painel publicado, o problema virou do usuário**

Com a coleta separada, o app segue no ar mostrando a última coleta boa.

- **Três camadas — as mesmas da Aula 6, com uma fonte nova**

coleta_web.py (raspa, à parte) → noticias.py (lê) → app.py (mostra).

O app.py da v5 NUNCA toca na rede para buscar notícia.

► A fragilidade da raspagem não pode virar fragilidade do produto.

O painel v5: duas fontes na mesma tela

Em cima, IBGE via API. Embaixo, imprensa via WebScraping — com o crédito da licença.

O resultado da Aula 7: o painel passa a ter DUAS fontes



E agora, ao vivo: publicar a v5

terminal

```
git add .
git commit -m "v5: coleta web da Agencia Brasil + secao de noticias"
git push

# .gitignore:
#   data/raw/*.html      NÃO sobe – conteúdo de terceiros, MB, regenerável
#   data/processed/      SOBE – é dele que o app publicado vive
```

Princípios que levamos

Quatro frases que sobrevivem à próxima biblioteca de scraping.

- **Pedir licença antes de pegar**

robots.txt, Crawl-delay, licença — nessa ordem, antes do código.

- **Baixar uma vez, processar muitas**

O snapshot é economia, educação e PROVA do que a página dizia.

- **Limpar é metade do trabalho**

Extrair é fácil; o que alimenta um painel é o dado limpo.

- **Coleta e uso são etapas separadas**

A raspagem vai quebrar; o produto não pode quebrar junto.



O caso, em 3 lições (v5)

O que a v5 ensinou, na ordem em que doeu.

- **A primeira pergunta do scraping não é técnica**
Duas de três fontes morreram em 'posso?', nenhuma por erro de código.
- **Você não extrai de uma página: extrai de uma ESTRUTURA que alguém escolheu**
Ela muda sem avisar, e o seu código para EM SILÊNCIO.
- **Restrição externa não é obstáculo a contornar: é requisito a incorporar**
Uma linha do robots.txt mudou a arquitetura do projeto inteiro.

► Na Aula 8: o CSV da SUA coleta entra no painel. Upload, cache, estado — e o TP2 fecha.